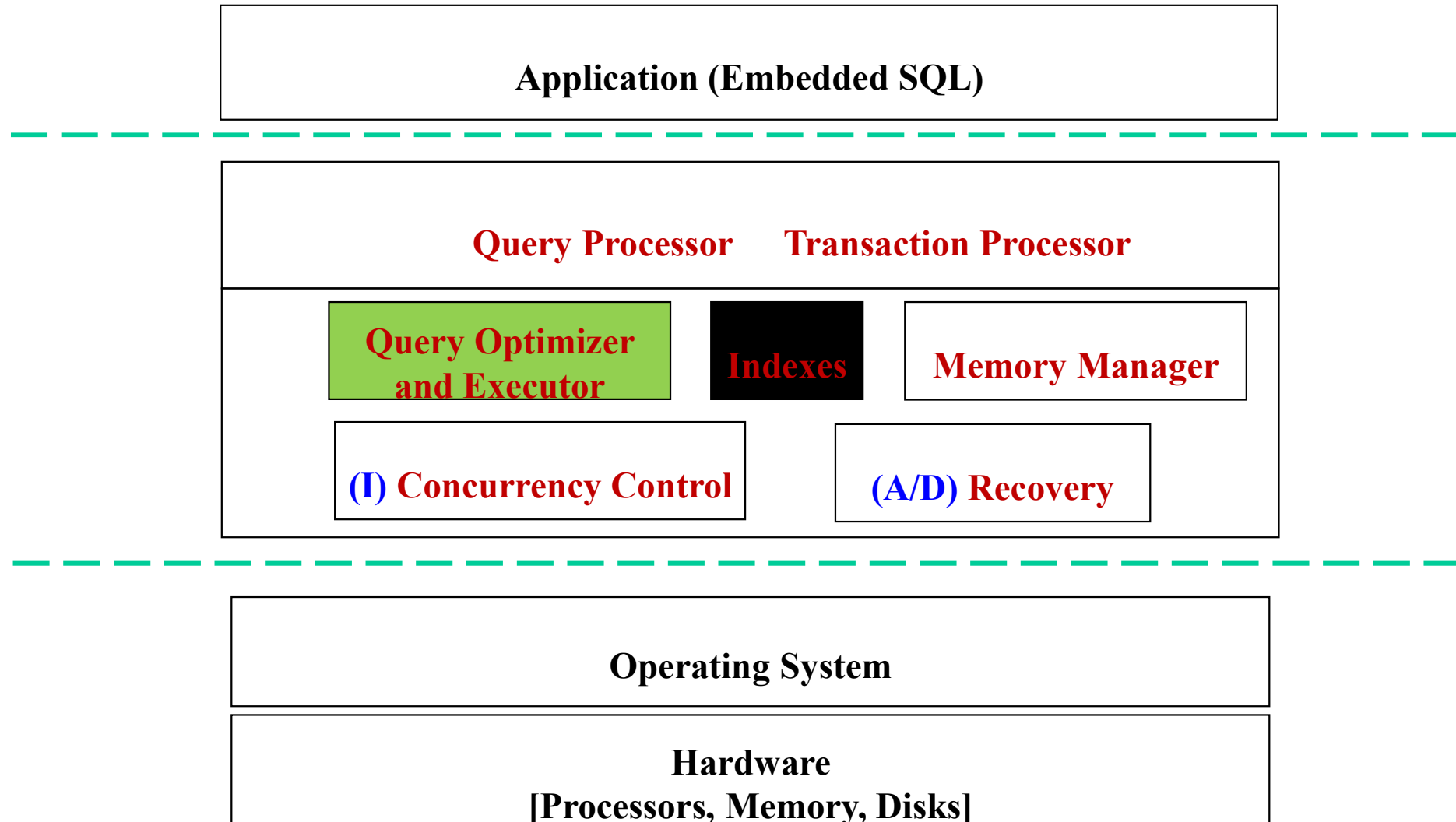


RDBMS Engine



LINEAR HASHING

E0 261

Jayant Haritsa

Computer Science and Automation

Indian Institute of Science



Indexing Techniques

- Comparison-based indexing (B⁺-trees)
- Computation-based indexing (Hashing)
- Main difference is that search cost in former is function of database size, in latter it is constant.
- But the former can support range searches, whereas the latter cannot.

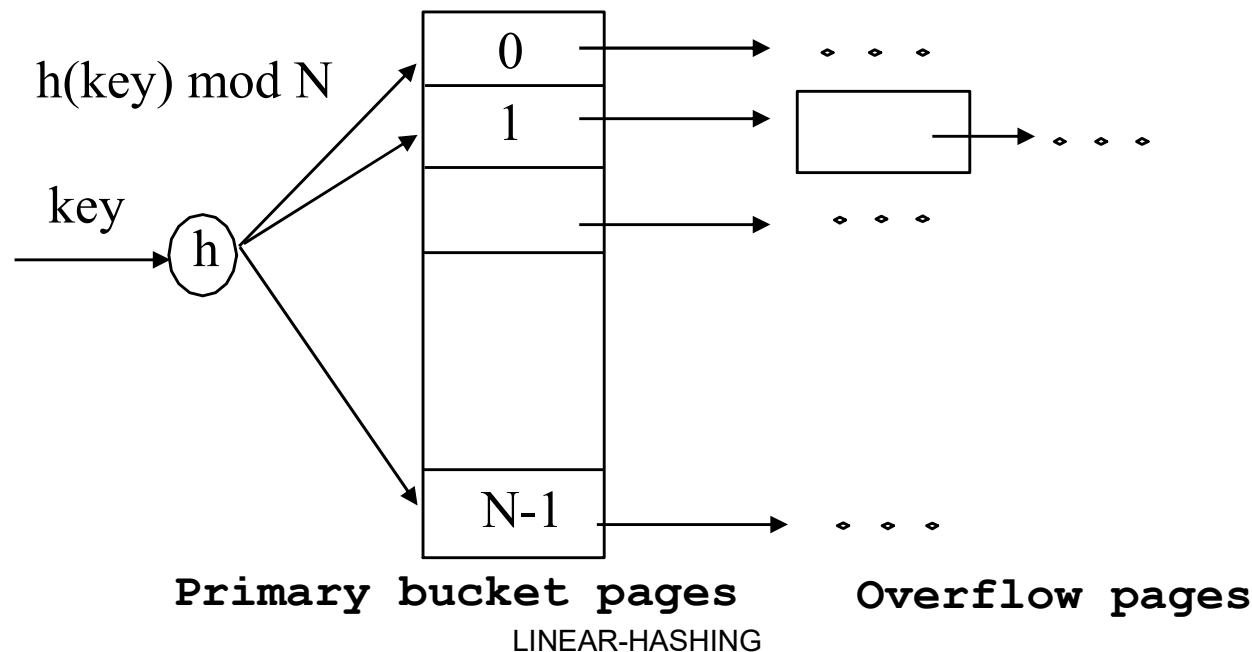


Hash-Based Indexes



Static Hashing

- # primary pages N fixed, allocated sequentially, never de-allocated; overflow pages if needed.
- $h(k) \bmod N = \text{bucket to which data entry with key } k \text{ belongs.}$



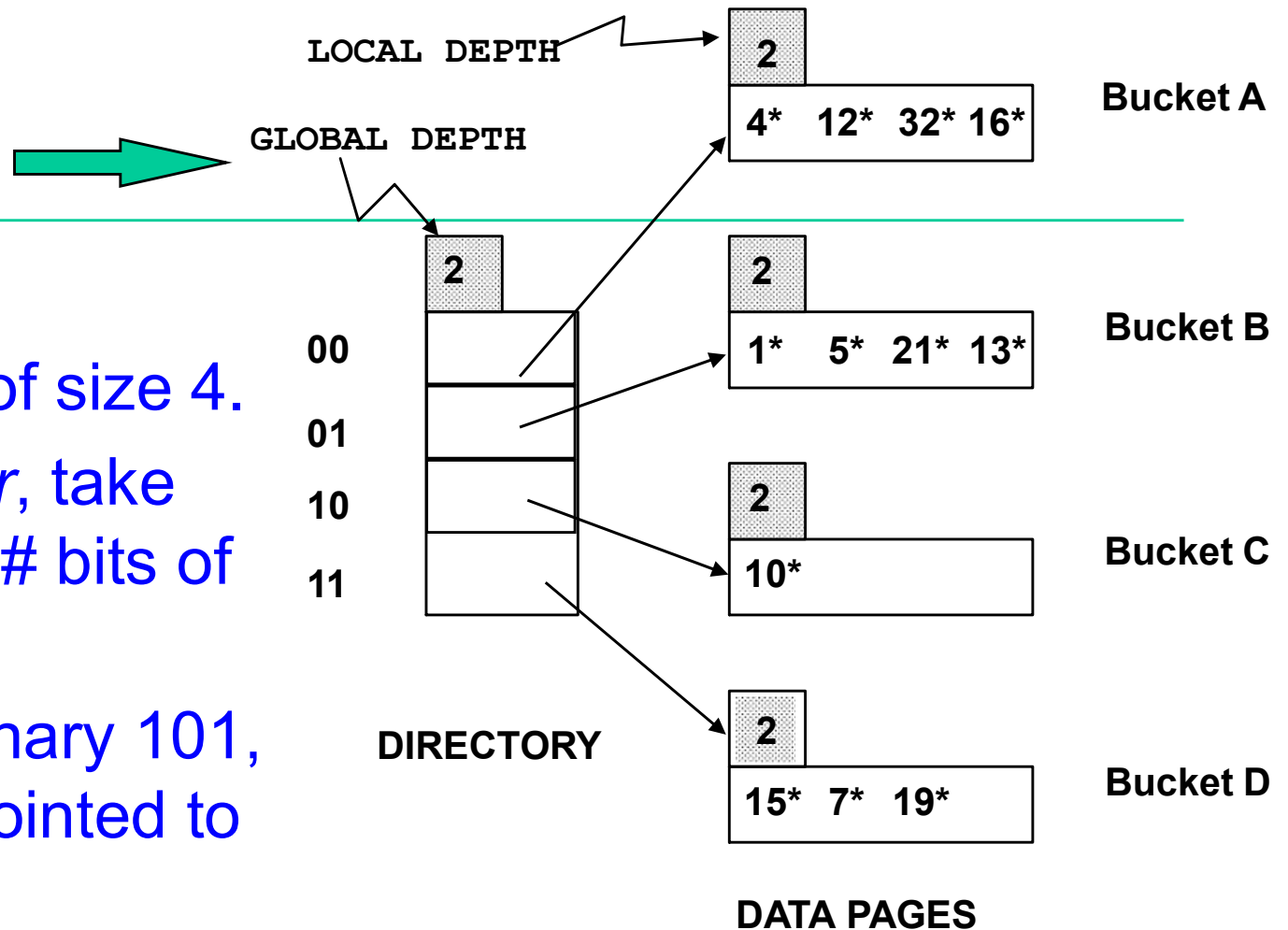
Extendible Hashing

- Situation: Bucket (primary page) becomes full.
Why not re-organize file by *doubling* # of buckets?
 - Reading and writing all pages is expensive!
 - Idea: Use directory of pointers to buckets, double # of buckets by *doubling the directory*, splitting just the bucket that overflowed.
 - Directory much smaller than file, so doubling it is much cheaper. Only one page of data entries is split. *No overflow page*.
 - Trick lies in how hash function is adjusted.



Example

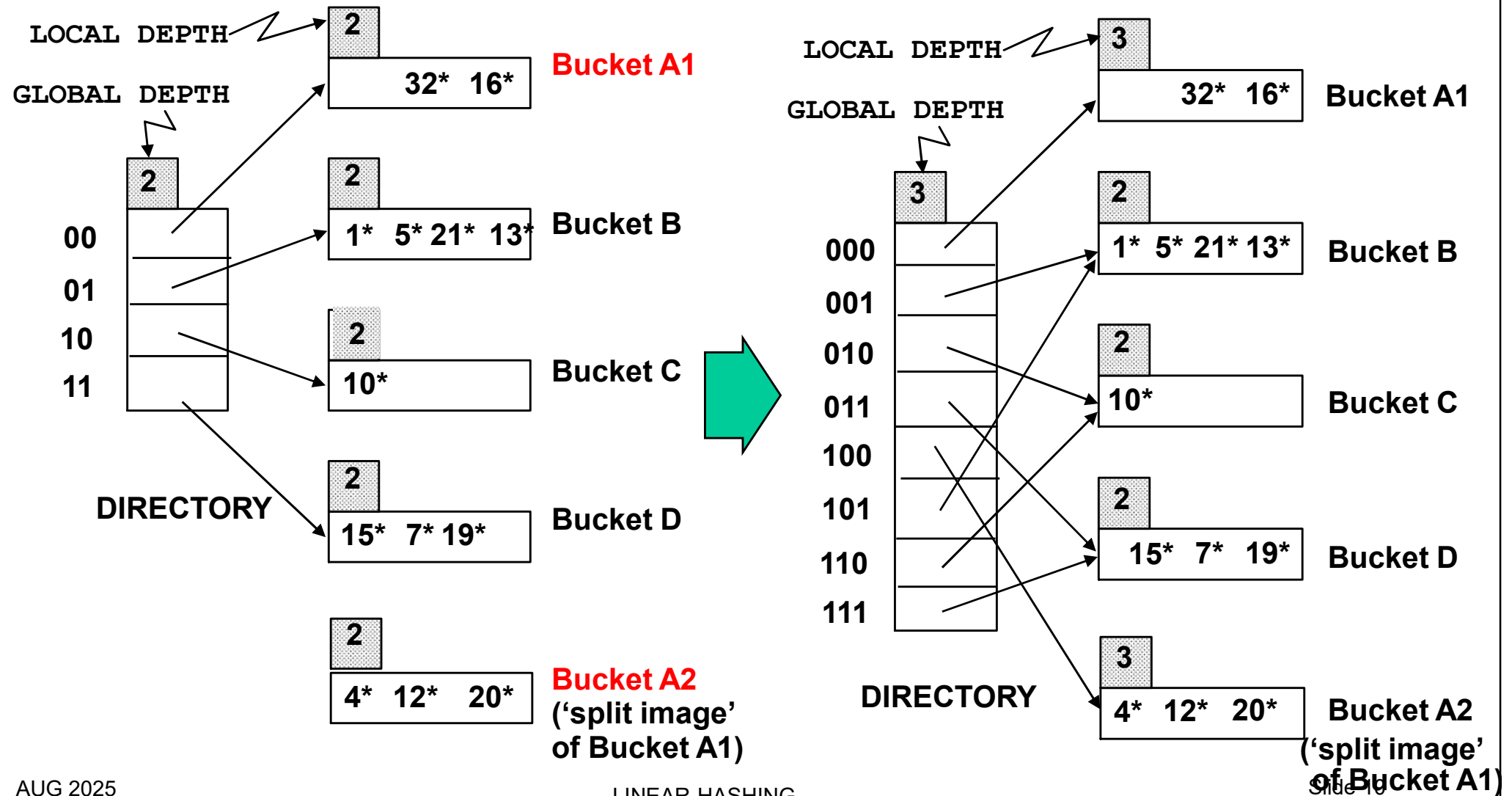
- Directory is array of size 4.
- To find bucket for r , take last '*global depth*' # bits of $h(k)$
 - If $h(k) = 5^* = \text{binary } 101$, it is in bucket pointed to by 01.



Insert: If bucket is full, *split* it (allocate new page, re-distribute).

If necessary, double the directory. (Splitting a bucket does not always require doubling; we can tell by comparing *global depth* with *local depth* for the split bucket.)

Insert $h(k)=20^*$ in Bucket A (Causes Doubling)



Comments on Extendible Hashing

- If directory fits in memory, equality search answered with one disk access; else two.
- Directory grows in spurts, and, if the distribution of *hash values* is skewed, directory can grow large.
- Multiple entries with same hash value cause problems
 - Unfixable issue for duplicates
 - Results in EH directory exploding!



Linear Hashing



Linear Hashing

- This is another dynamic hashing scheme, an alternative to Extendible Hashing.
- LH handles the problem of long overflow chains without using a directory, and handles duplicates.
- Idea: Use a family of hash functions $h_0, h_1, h_2, \dots$
 - $h_i(\text{key}) = h(\text{key}) \bmod(2^i N)$; N = initial # buckets
 - h is some hash function (range is 0 to $2^{|\text{MachineBitLength}|}$)
 - If $N = 2^{d_0}$, for some d_0 , h_i consists of applying h and looking at the last d_i bits, where $d_i = d_0 + i$.
 - h_{i+1} doubles the range of h_i (similar to directory doubling)

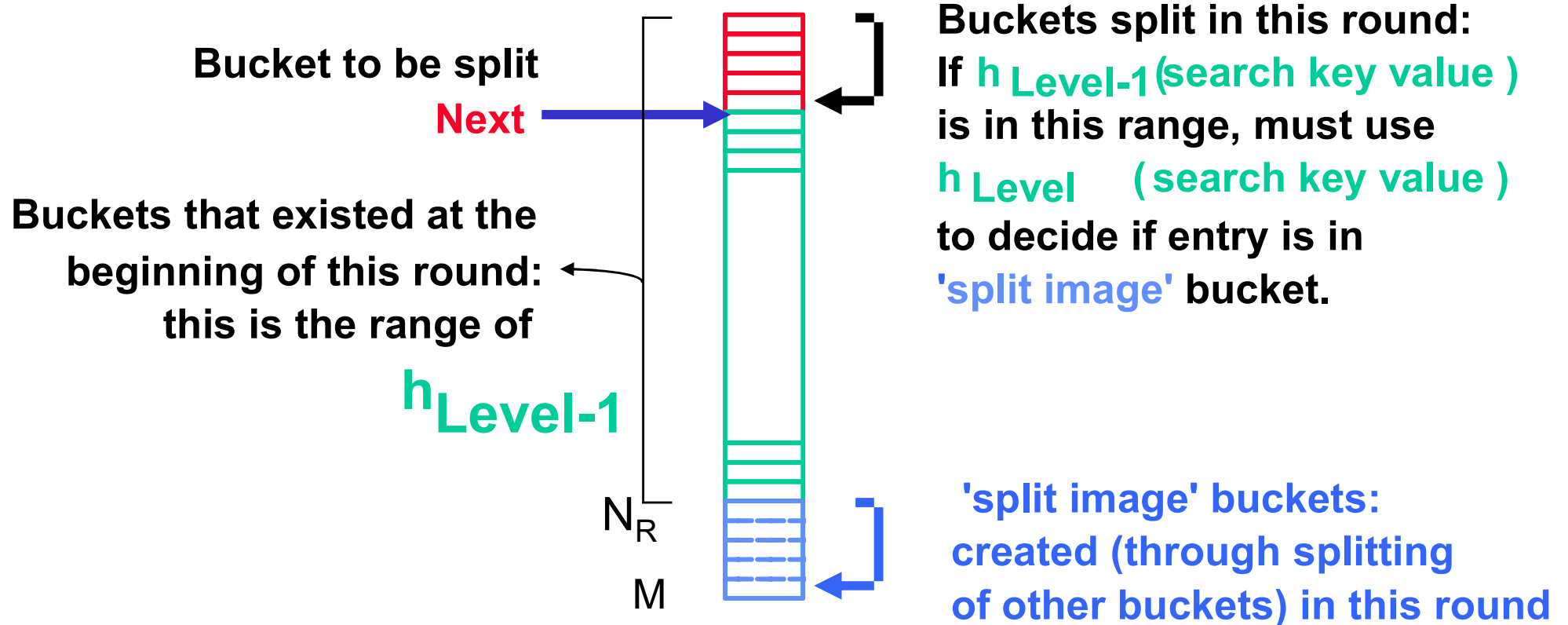
Linear Hashing (Contd.)

- Directory avoided in LH by using overflow pages, and choosing bucket to split round-robin.
 - Splitting proceeds in 'rounds'. Round ends when all N_R initial (for round R) buckets are split.
 - Buckets 0 to $Next-1$ have been split; $Next$ to N_R yet to be split.
 - Current round number is $Level$.



Overview of LH File

- In the middle of a round.



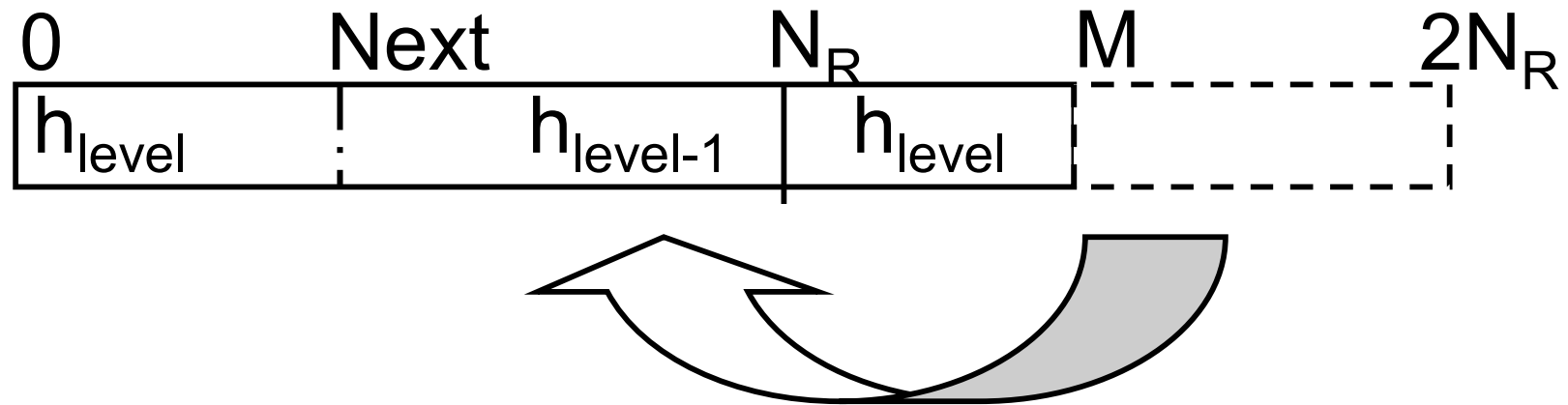
LH Search

To find bucket for data entry k , find $h_{Level-1}(k)$:

- If $h_{Level-1}(k)$ in range '*Next to N_R* ', k belongs here.
- Else, k could belong to bucket $h_{Level-1}(k)$ or bucket $h_{Level-1}(k) + N_R$; must apply $h_{Level}(k)$ to find out.



Simpler Formulation



- $m = h_{\text{level}}(k)$
if $m \geq M$, $m = m - N_R$

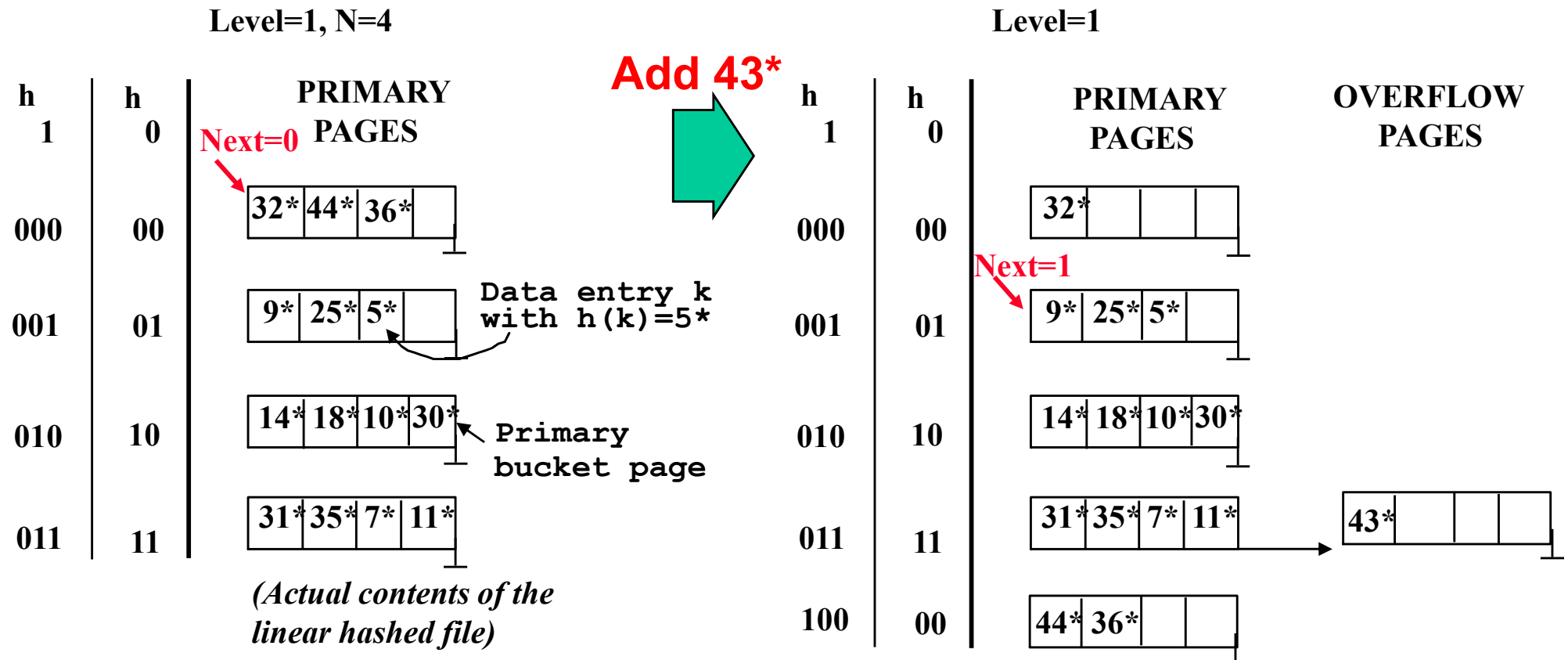
LH Insert

- Find bucket by applying $h_{Level-1} / h_{Level}$:
 - If bucket to insert into is full:
 - Add overflow page and insert data entry.
 - (Maybe) Split *Next* bucket and increment *Next*.
- Can choose any criterion to 'trigger' split.
- Since buckets are split round-robin, long overflow chains don't develop!
- Doubling of directory in Extendible Hashing is similar; switching of hash functions is *implicit* in how the # of bits examined is increased.

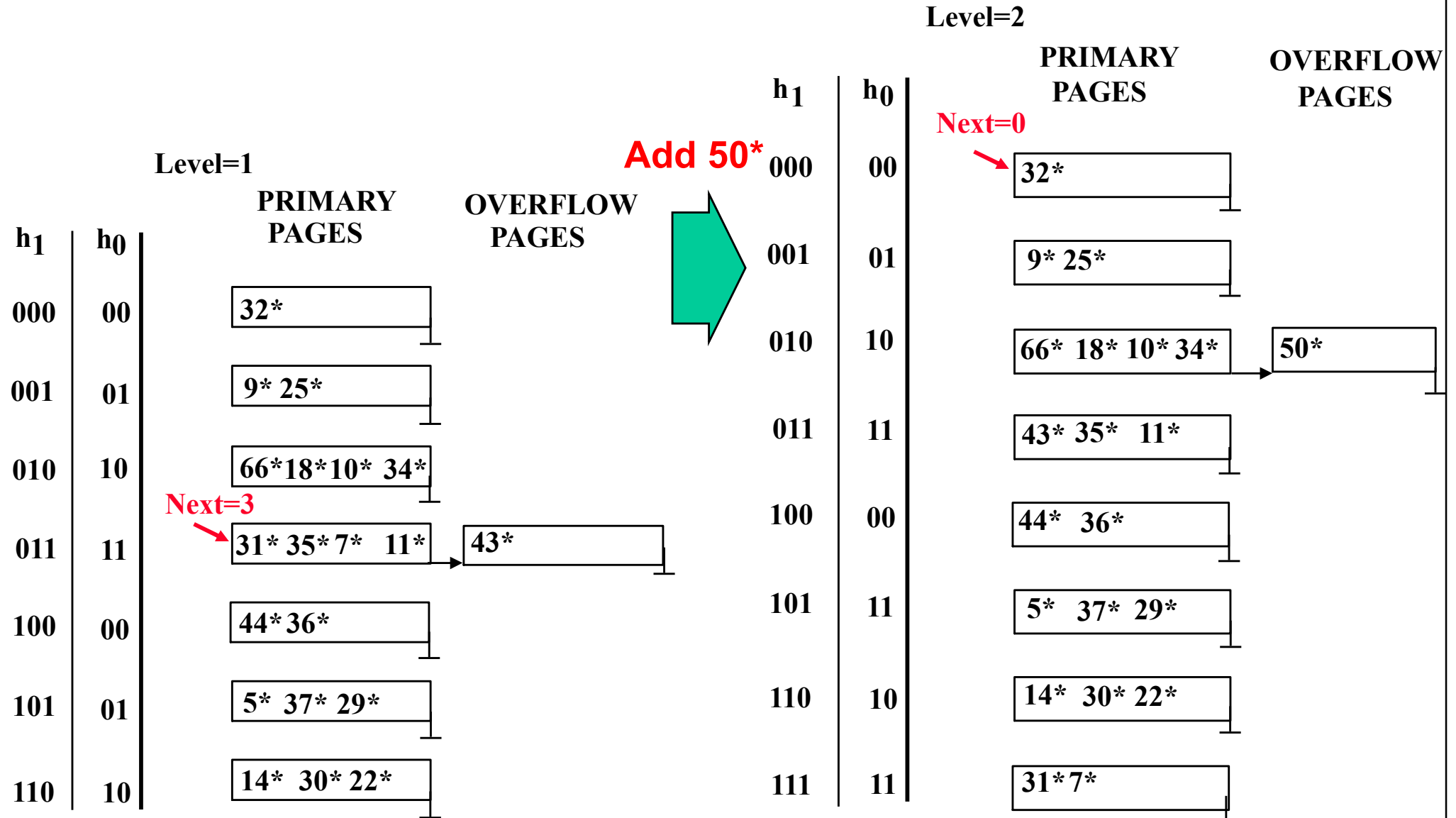


Example of Linear Hashing

- On split, h_{Level} is used to re-distribute entries.



Example: End of a Round



Physical Address Computing

- Logical bucket address given by hashing must be converted into the physical address of the bucket on disk.
- Simple solution: contiguous allocation on disk, but not feasible in general
- In practice, keep giving larger and larger contiguous chunks as the file size grows (Equation 6 doubles chunk size with $K=1$)



Points to Note

- Hash Function:
 - $h_i(k) = [(A k) \bmod w] \bmod 2^i N$
where $A = 6125423371$ $w = 2^{32}$
(A is prime w.r.t. w)
- In LH, split is local impact, whereas in the B-tree, split is “global”
- In LH, duplicates don't cause a problem because of presence of overflow buckets

Problem

- For small cardinality domains (e.g. gender), neither hashing nor indexing work
- Solution: Bitmap Indexes



END LINEAR-HASHING

E0 361

